

1 General Description

This document details the theory and use of PLX's NET2272 with Virtual Endpoint Technology™. By applying virtual endpoints, the NET2272 can have as many endpoints as the USB specification allows (up to 15 IN and 15 OUT). Multiple endpoints make possible multiple independent data streams between USB and the local bus of the NET2272. This means that a slow piece of data in one endpoint will not block the progress of another piece of data in a different endpoint. Virtual endpoints are ideal for cost-sensitive devices that require several interfaces such as multi-function devices or devices with data streams of differing speeds.

Physically the NET2272 incorporates three endpoints between the USB and local bus. Virtual Endpoint Technology™ allows mapping of any number of virtual endpoints onto a physical endpoint (see figure 1). This Virtual to physical endpoint mapping is controlled by firmware. It means that a local processor and memory is required to manage the virtual endpoints.

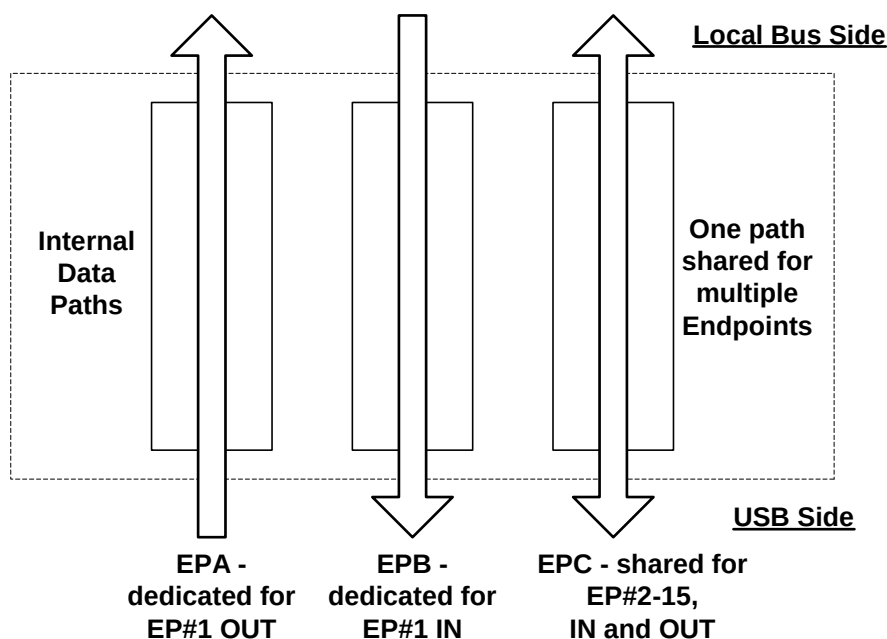


Figure #1: Virtual Endpoint Usage Example

2 Theory of Operation

Normally, when the USB Host attempts to access an unassigned endpoint, the NET2272 responds with a timeout. The Host considers a timeout response to be a fatal error. The Host may retry a transaction with a fatal error, but after a fixed number of retries, the host shuts down that endpoint.

With virtual endpoints enabled, the NET2272 responds with a NAK instead of a timeout. The NAK causes the USB Host to retry the request until it gets an ACK from the NET2272. (This will not work for Isochronous packets as the USB Host does not retry those.)

As the NAK is being sent to the USB Host, a virtual endpoint interrupt is generated to the local processor. The processor can then reassign one of the NET2272's existing endpoints to the newly requested endpoint. That new endpoint will then be available when the Host returns to retry the original request.

Before reassigning an endpoint, it must first be disabled. Once disabled, the local processor stores the endpoint's data and state information and then loads in the newly requested endpoint's data and state information. The physical endpoint can then be reassigned and re-enabled (see figure 2).

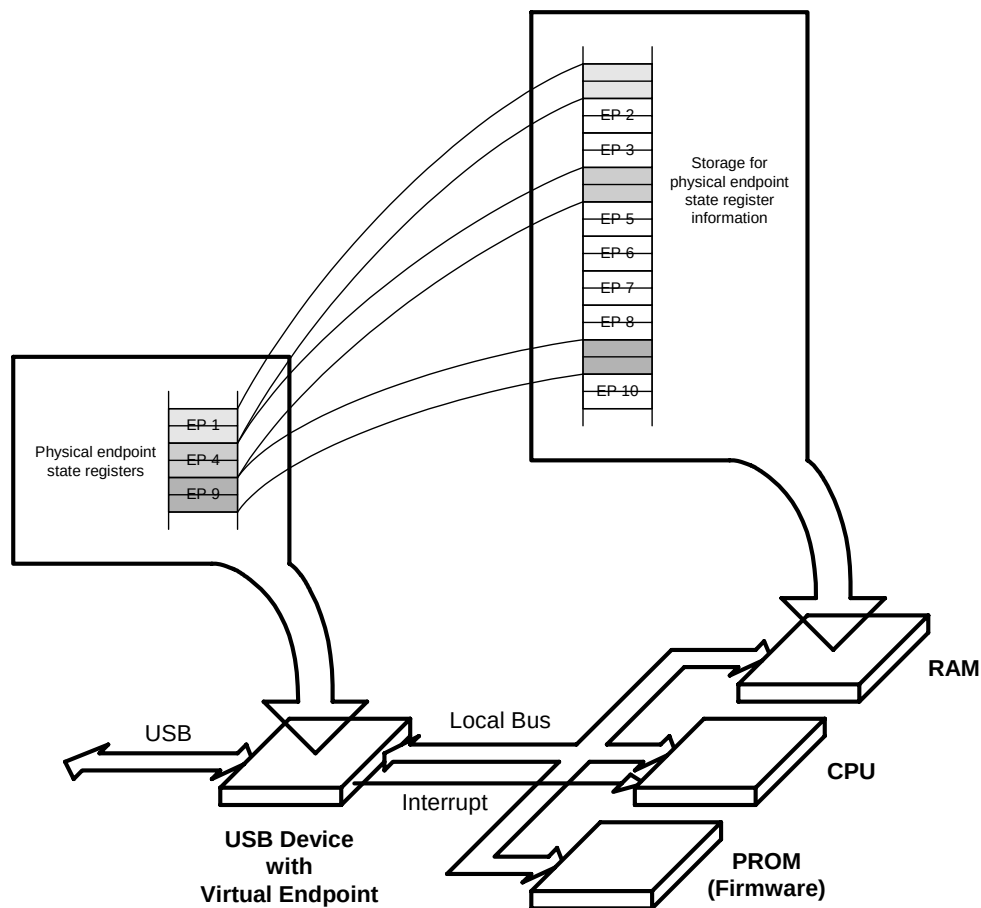


Figure #2: Storing an Endpoint's State in Memory

Virtual Endpoint participation is completely flexible, all physical endpoints (excluding endpoint 0) may participate in Virtual Endpoint re-assignment, or some physical endpoints can be dedicated to specific high-usage endpoints.

As a firmware based save and restore system is required for changing the endpoint associated with a physical data path, overall performance is proportional to the switching frequency.

3 Relevant Registers

To operate the NET2272 with virtual endpoints, the following registers need to be used / monitored.

Register Name	Bits	Set to	Description
USBCTL1	Virtual Endpoint Enable	1	This enables the NET2272's virtual endpoint function
IRQENB0	Virtualized Endpoint Interrupt Enable	1	Enables a local interrupt to trigger when an IN or OUT token is received by a virtualized endpoint
IRQSTAT0	Virtualized Endpoint Interrupt	Output	Set when one of the virtual endpoint interrupts (VIRTOUT / VIRTIN) are set
VIRTOUT0/ 1	Virtual OUT Interrupts	Output	Indicates which virtual endpoint number received an OUT token
VIRTIN0/ 1	Virtual IN Interrupts	Output	Indicates which virtual endpoint number received an IN token
EP_CFG	Endpoint Enable	Varies	The physical endpoint must be disabled before its endpoint number can be changed
EP_CFG	Endpoint Number	Varies	Assign a logical endpoint number (1-15) to a physical endpoint
EP_CFG	Endpoint Direction	Varies	Selects whether a physical endpoint should be an IN or OUT endpoint.
EP_BUFF_STATE	Buffer A State Buffer B State	Output	Indicates the full / empty state of the internal buffers. It shows if there is data still held in the internal buffers that still needs to be transferred.
EP_DATA	Endpoint Data	Varies	Any data still in the internal buffers must be read out through this register before the endpoint number can be changed
EP_CFG EP_IRQENB EP_TRANSFER RP_RSPSET EP_MAXPKT	Endpoint Status Bits	Varies	The complete state of the current endpoint must be saved before loading in the state of the requested endpoint

Table #1: Virtual Endpoint Configuration

4 Switching Virtual IN Endpoints

Let's take the example where physical endpoint A (EPA) is being used as a virtual endpoint. Initially, EPA might be set up as USB endpoint 1 in the IN direction (EP_81). In this case, any IN packets to EP_81 will transfer normally, as will packets for any of the other already-mapped endpoints (for instance an OUT to EP_01 mapped to EPB or an IN to EP_84 mapped to EPC).

When an IN token arrives for an unmapped endpoint, let's say EP_82, the NET2272 responds to the USB Host with a NAK. This tells the Host to retry the same transaction again later. At this time, the NET2272 also sends an interrupt to the local processor. The local processor determines the source of the interrupt by reading the IRQSTAT registers. The VIRTOUT and VIRTIN registers indicate exactly which virtual endpoint is being requested. In our example, they indicate that the USB Host requested an IN transfer from EP_82 because bit 2 of the VIRTIN0 register is set.

As an IN endpoint, firmware must first stop sending data to the Host through EPA. EPA then needs to be disabled. Firmware waits for EPA's enable bit to clear (the endpoint will clear its enable bit within a single packet transfer time). If there is any un-transferred data remaining in the endpoint buffers (as indicated by EP_BUFF_STATE), firmware changes the Endpoint Direction and reads out that data from the EP_DATA register. This data, along with the state of the endpoint registers (EP_CFG, EP_IRQENB, EP_TRANSFER, EP_RSPSET, and EP_MAXPKT) is saved as an "endpoint context" in local memory.

Now firmware can change EPA's Endpoint Number and Endpoint Direction to 0x82 (the direction can be cleared if the new request was for an OUT transfer instead of an IN transfer); clear the virtual interrupt (VIRTIN); restore any context switch information saved for EP_82; and finally enable EPA again (see figure 3).

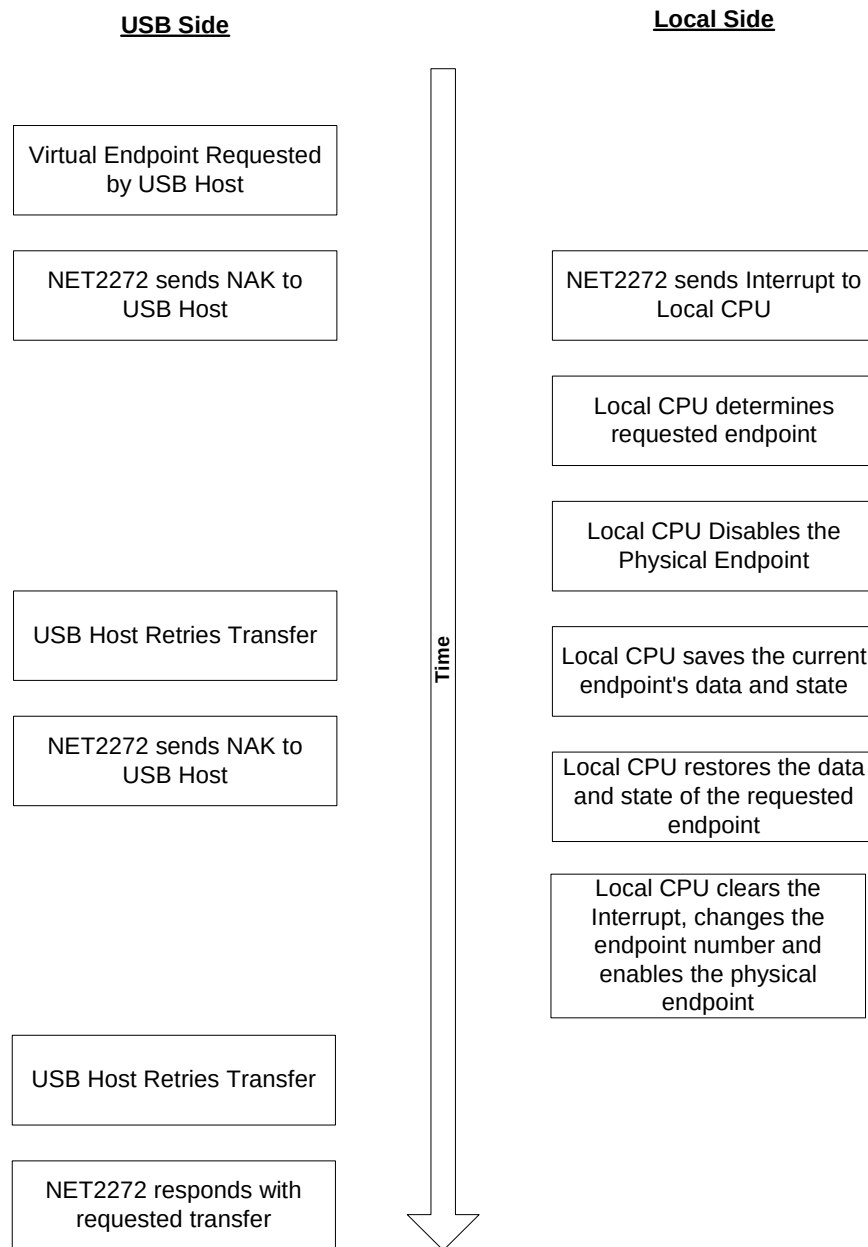


Figure #3: Endpoint Switching Sequence

5 Switching Virtual OUT Endpoints

Let's take the example where physical endpoint EPB is being used as a virtual endpoint. Initially, EPB might be set up as USB endpoint one in the OUT direction (EP_01). In this case, any OUT packets to EP_01 will proceed normally, as will packets for any other already-mapped endpoints (for instance an IN to EP_81 mapped to EPB or an IN to EP_84 mapped to EPC).

When an OUT token arrives for an unmapped endpoint, let's say USB EP_02, the NET2272 responds to the USB Host with a NAK. This tells the Host to retry the same transaction again later. At this time, the NET2272 also sends an interrupt to the local processor. The local processor determines the source of the interrupt by reading the IRQSTAT registers. The VIRTOUT and VIRTIN registers indicate exactly which virtual endpoint is being requested. In our example, they indicate that the USB Host requested an OUT transfer to EP_02 because bit 2 of the VIRTOUT0 register is set.

As an OUT endpoint, EPB can be disabled immediately. Firmware waits for the enable bit to clear (the endpoint will clear its enable bit within one packet transfer time). If there is any data still in the internal buffers (as indicated by EP_BUFF_STATE), that data should be read out from the EP_DATA register using the normal OUT data transfer function. Firmware then saves the state of endpoint registers (EP_CFG, EP_IRQENB, EP_TRANSFER, EP_RSPSET, and EP_MAXPKT) to an endpoint context in local memory.

Now firmware can change EPB's Endpoint Number and Endpoint Direction to 0x02 (the direction should be changed if the new request was for an IN transfer instead of an OUT transfer); clear the virtual interrupt (VIRTOUT); restore any saved context information for EP_02; and finally enable EPB again.

6 Summary

Virtual Endpoint Technology™ is made possible by dynamically assigning different USB endpoint numbers to the physical endpoints of the NET2272. The NET2272 allows changes to endpoint parameters without loss of data or endpoint state. To the USB Host, it appears that all USB endpoints are available, only some of them are temporarily busy. This is useful for any application that utilizes multiple independent data streams.

There are some limitations to the Endpoint switching:

- A processor and memory must be available on the local bus to manage the switching.
- Isochronous endpoints cannot be virtualized as the USB Host does not automatically retry NAK'd Isochronous transfers.
- As a limited set of physical hardware is being shared, overall performance depends on switching frequency.

But even with these limitations, virtual endpoints are still ideal for designs that utilize multiple endpoints with only a couple of high-usage channels required to be available at all times.